

The Three-Layer Architectural Model as Integrating Frame for the Layer Distinctions in the Coordination Knowledge Substrate Pattern

Author: Wenxin Li (Independent Researcher)

ORCID: 0009-0004-8065-3235

Date: May 5, 2026

License: Creative Commons Attribution 4.0 International (CC BY 4.0)

Attribution

This work derives from and formalizes the Coordination Knowledge Substrate (CKS) pattern introduced in *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems* (Li, April 2026). It does not introduce new axioms. Its sole contribution is to formalize the three-architectural-layers integrating frame that organizes the layer distinctions of the parent foundational note (A1.15: *Layer Distinctions: Why CKS Is Not a Workflow Engine, an Agent Framework, or a Control Plane*) — articulating each layer's commitment, the load-bearing distinguishing commitment of each layer relative to CKS, and the relationship between the layer treatment and the broader hybrid-composition framework — so that the four standalone specializations that follow (A2.87 on workflow engines, A2.88 on agent frameworks, A2.89 on control planes, A2.90 on the composition pattern) can be defended, implemented, and tested against a publicly fixed integrating frame.

Abstract

The CKS parent foundational note A1.15 commits to three layer distinctions — that CKS is not a workflow engine, not an agent framework, and not a control plane — inside a single integrated text. The full operational decomposition of A1.15 splits into five derivation notes: an integrating frame (this note) and four specializations covering each of the three layer distinctions and the composition pattern across them. This note specifies the integrating frame: a three-layer architectural model in which the *coordination-knowledge layer* is where coordination state lives (decisions, authority, rationale, preserved conflict; CKS commits here), the *coordination-mechanism layer* is where execution behavior lives (step sequences, parallelism, multi-agent communication, the LLM execution machinery; workflow engines and agent frameworks commit here), and the *control-plane layer* is where infrastructure operations live (lifecycle, identity, access control, observability; Stevens-style primitives commit here). The note defines each layer, names the three layer distinctions and the composition pattern at the integrating-frame level (with full operational treatment delegated to A2.87–A2.90), states what the integrating frame does not claim, and provides an operational test at the integrating-frame level. The treatment distinguishes A1.15's orchestration-layer adjacency framing from A1.14's memory-location adjacency framing, and names the extension-vs-recapitulation relationship the four specializations carry to the source paper.

1. Why the integrating frame needs to be formalized as standalone

The parent foundational note A1.15 commits to three layer distinctions inside a single integrated text. The full operational decomposition treats each distinction (and the composition pattern across them) as a standalone derivation. Four such notes — A2.87, A2.88, A2.89, A2.90 — specialize the parent. This note is the fifth: the integrating frame the four specializations share.

The motivating cases. Deployments where CKS coordinates with one or more orchestration-layer objects are common and operationally varied: a BPMN-based workflow engine triggering CKS cells, a multi-agent framework executing CKS cells, a control plane authorizing substrate access. Each scenario requires the layer model to be operationally specified before the per-distinction specializations can be defended; without the integrating frame, the specializations risk reading as four loosely related arguments rather than as four specializations of one architectural commitment.

The strategic prior-art posture. Patentable derivations addressing AI-coordination architectures involving workflow, agent, or control-plane integration are more defensibly contested when the three architectural layers are publicly formalized as a single integrating object — independently of the per-distinction specializations.

The connection to A1.14 and A1.16. A1.14 specifies *memory-location* adjacencies (where knowledge is stored relative to the LLM); A1.15 specifies *orchestration-layer* adjacencies (what architectural layer the design object commits at). The two foundational notes together specify CKS's full architectural position along two distinct axes. A1.16 specifies how CKS composes with non-CKS objects through three patterns (input to a cell, derivative view of substrate, separate concern); A1.15's layer distinctions specify which layer each non-CKS object commits at, making A1.16's patterns operationally applicable to specific orchestration-layer objects.

2. The coordination-knowledge layer

The **coordination-knowledge layer** is the architectural layer where coordination state lives — what was decided, by whom, under what authority, with what rationale, and what contradictions remain unresolved. The CKS substrate operates here. Its commitments are about state, and they are five.

(a) *Coordination state is persistent.* Substrate content per A2.08's substrate-layer commitments persists across cell runs, sessions, and operational events. The state is not transient to a process instance, an agent run, or an operational concern.

(b) *Coordination state is conflict-preserving.* Per A1.03 and the conflict-as-first-class decomposition (A2.13–A2.17), contradictions remain as substrate state with provenance per A2.16; they are not reconciled away by infrastructure or absorbed into engineered consistency.

(c) *Coordination state is human-governed in the authority sense.* Per A1.01 and the human-governance decomposition (A2.01–A2.07), the three rights — to inspect, to modify, and to override substrate content and orchestration rules at any time — are exercisable over substrate content. Authority structure per A2.47 is itself substrate content, not metadata about substrate content.

(d) *Coordination state is addressable across sessions.* Per A2.59 (Guarantee C: substrate changes are addressable), substrate content has identifiers that persist beyond the lifecycle of any process, agent, or operational object. Addressability is a property of the layer, not of a particular execution.

(e) *Cell behavior follows orchestration rules.* Per A2.04, cell-level behavior is human-authored at the rule level, with cells executing those rules at runtime under the AI-as-substrate-mediator commitment per A1.04 and its decomposition (A2.18–A2.23). Cell behavior is what the layer commits to *running*; substrate content is what the layer commits to *carrying*.

The coordination-knowledge layer is what CKS commits at. The other two layers commit elsewhere; their commitments do not reach into this layer's content.

3. The coordination-mechanism layer

The **coordination-mechanism layer** is the architectural layer where work executes — where steps run, routing happens, retries are triggered, parallelism is managed, and (for agent frameworks) multi-agent communication and tool calling are structured. Workflow engines and agent frameworks operate here. Its commitments are about behavior, and they are four.

(a) *Step sequences are defined and enforced.* Workflow engines execute multi-step processes against process specifications (BPMN, DSL, code-defined orchestrators); agent frameworks execute multi-agent reasoning against agent definitions. The architectural commitment is to the execution mechanics — sequencing, transitions, branching, retries, and parallel coordination that make multi-step work runnable.

(b) *Execution state is persisted at the layer's timescale.* Workflow engines persist execution state for the duration of a process instance (which step ran, with what inputs, outputs, and errors); agent frameworks persist execution state for the duration of an agent run (agent histories, message logs, tool-call records, framework-managed memory stores). Persistence is at the mechanism layer; coordination state at the CKS layer persists across these lifecycles.

(c) *LLM execution machinery is included for agent frameworks.* Prompting, tool calling, multi-step reasoning, and multi-agent communication all sit at the coordination-mechanism layer when an agent framework is the executing object. Source paper §4.4 treats Multi-Agent Coordination Infrastructure (MACI) as the canonical 2024–2026 example of a coordination-mechanism-layer commitment in the multi-agent direction; A2.88 extends that framing to the broader class of agent-framework objects.

(d) *Behavior — not state — is the architectural commitment.* Workflow engines and agent frameworks address how work executes; they do not address what coordination state the work produces beyond their own execution records. A workflow engine can run a process for years and faithfully record every step that ran without ever answering *what was the rationale for the decision this process implements*.

The coordination-mechanism layer collapses §4.3's two-layer rendering of reasoning execution and coordination mechanisms; the collapse is for expository convenience and relocates no commitment.

4. The control-plane layer

The **control-plane layer** is the architectural layer where infrastructure-level concerns are managed — process lifecycle, resource allocation, identity, access control, observability, and the gateway-layer policies that make agent operation governable in the operational sense. Control-plane primitives, in the sense Stevens uses the term in his work on trustworthy AI agents (cited at source paper §8.1), operate here. Its commitments are about operations, and they are five.

(a) Process lifecycle is managed. When processes start, run, scale, and terminate at the runtime level — including container orchestration, scheduling, and runtime supervision.

(b) Resource allocation is managed. Compute, storage, network, and other resources are provisioned, tracked, and reclaimed.

(c) Identity and access control are managed. Who is authenticated to which environment, who is permitted to invoke which capability, under which conditions, with what credentials.

(d) Observability is managed. Runtime logs, metrics, traces, and audit trails at the operational level — what services were called, when, with what latency, against which dependencies.

(e) Gateway-layer policies make agent operation governable in the operational sense. Approval gates, reviewer workflows, accountable decision logs, and break-glass override at the operational layer — the primitives Stevens names as making autonomous-agent deployment operationally trustworthy.

Per source paper §8.1, CKS is complementary to control-plane primitives: the control plane provides the infrastructure that makes a substrate and its cells operationally viable, and the substrate provides the coordination knowledge the running system operates on. The two layers compose; neither substitutes for the other.

5. The three layer distinctions, named at the integrating-frame level

Each of the three layer distinctions has a load-bearing CKS-distinguishing commitment. This section names each at the integrating-frame level; A2.87, A2.88, and A2.89 specialize each as a standalone derivation.

Layer Distinction 1 — CKS is not a workflow engine. Workflow engines persist *execution state* (which step ran, with what inputs, outputs, and errors), not coordination-knowledge state. The most common conflation treats workflow execution state as if it were coordination-knowledge state — execution logs are accountability traces in a narrow sense (what the engine did), but not the trace CKS specifies per A1.07 and A2.35–A2.41 (what was decided, under what authority, with what rationale). The CKS-distinguishing commitment is the coordination-state architecture per §2.

Layer Distinction 2 — CKS is not an agent framework. Agent frameworks manage the LLM execution machinery and multi-agent coordination at the coordination-mechanism layer; they hold execution state for the duration of an agent run, while CKS substrate holds coordination state across cell runs through the human-governed substrate per A1.01. The most common conflation treats agent-framework memory as if it were coordination-knowledge state. The CKS-distinguishing commitment is the

AI-as-substrate-mediator framing per A1.04 versus the autonomous-reasoning-agent framing.

Layer Distinction 3 — CKS is not a control plane. Control planes manage infrastructure-level concerns; CKS substrates carry coordination state. The most common conflation treats control-plane authority decisions (who can call which API, who is authenticated to which environment) as if they were CKS authority decisions per A2.01–A2.03 (the inspect, modify, and override rights formalized in the human-governed commitment). A user with full control-plane access to the host might still not be a CKS governor; conversely, a CKS governor must hold control-plane access sufficient to exercise their rights. Control-plane access is necessary, but not sufficient, for CKS governance.

The four specializations carry distinct citation relationships to the source paper. The workflow-engine treatment (A2.87) and the agent-framework treatment (A2.88) *extend* the source paper's layer-distinction framing — §2.1 (the substrate-vs-cell split) and §4.4 (MACI as the canonical coordination-mechanism-layer object) — to design-object families the source paper does not directly name. The control-plane treatment (A2.89) *recapitulates* §4.3 and §8.1 directly without extension. A2.90 specializes the composition pattern across all three.

6. The composition pattern, named at the integrating-frame level

The three layers compose. A complete deployment may use all three; each layer's role is distinct; the substrate's commitments are not delegated to or claimed from the others. Three composition patterns are operationally common at the integrating-frame level; A2.90 specializes the composition pattern as a standalone derivation.

A workflow engine triggers a CKS cell. The engine decides when the cell should run, given the workflow's process state and triggering conditions; the cell reads from and writes to the substrate under human-authored orchestration rules. Two records exist after the cell runs: the engine records that it ran the cell; the substrate records what the cell decided, under what authority, with what rationale, against what conflicts.

An agent framework executes a CKS cell. The framework manages the LLM execution machinery; the cell's role is the same as above. Framework memory holds execution state for the duration of the cell's run; the substrate holds coordination state across cell runs.

A control plane authorizes substrate access. The control plane authenticates the user and authorizes host-level access; the substrate enforces CKS-level governance authority once that access is granted. Control-plane access is necessary for CKS governance to be exercisable, and CKS governance is the layer at which the substrate's content-level authority decisions are made.

Across all three patterns, the substrate is the only layer whose state persists by architectural commitment across the lifecycles of the other layers' objects. The composition aligns with A1.16: the workflow-trigger and agent-execute patterns instantiate A1.16's *input-to-a-cell* composition, while the control-plane-authorization pattern instantiates A1.16's *separate-concern* composition. A2.90 specializes the layer-aware composition standalone.

7. What the integrating frame does NOT claim

Stating precisely what the integrating frame does not claim keeps the standalone framing from drifting beyond what the source paper supports.

(a) Not all CKS deployments must include all three layers. A deployment may have CKS without a workflow engine, without an agent framework, or with a minimal control plane. The architectural commitment is that when these objects are present, they operate at their respective layers without absorbing CKS's commitments.

(b) The frame does not foreclose other orchestration-layer objects. The three named here are the most common conflationations in the source paper and in 2024–2026 AI coordination discussions; future work may identify additional orchestration-layer objects warranting their own layer-distinction treatment.

(c) The frame does not specify implementation patterns for layer coordination. Specific composition primitives — cell-triggering APIs, substrate-aware agent memory interfaces, control-plane policies that read substrate content — are explicitly future work per source paper §13.3.

(d) The frame does not claim that any layer is more important than the others. Each is responsible for state, behavior, or operations at its own architectural scope; the integrating treatment is about how the layers fit together, not about layer priority.

(e) The frame does not require that the three layers be implemented by separate vendors or technologies. A single deployment may include CKS, workflow capabilities, and control-plane features within one technology stack; the architectural commitment is to the layer distinctions being preserved in whatever implementation is used.

(f) The frame does not foreclose architectural drift. Implementations under pressure may drift toward conflating layers — using workflow execution state as coordination state, treating agent-framework memory as a substrate substitute, equating control-plane access with CKS authority. The layer distinctions are what makes the drift recognizable as architectural deviation rather than as natural evolution.

8. Operational test at the integrating-frame level

A system instantiates the three architectural layers at the integrated level if and only if all of the following are true at all times during the substrate's existence:

- 1 Coordination-knowledge state — decisions, authority, rationale, and preserved conflicts — lives in a substrate per §2, not in workflow execution logs, agent-framework memory, or control-plane state.
- 2 Execution behavior — step sequences, retries, parallelism, multi-agent coordination, tool routing, and the LLM execution machinery — lives in workflow engines or agent frameworks where appropriate per §3, not in the substrate.

- 3 Infrastructure concerns — process lifecycle, identity, access control, resource allocation, and operational observability — live in the control plane per §4, not in the substrate.
- 4 The three layers compose per the patterns named in §6: a complete deployment may use all three, but each layer's role is distinct, the substrate's commitments are not delegated to the other layers, and the other layers' commitments are not claimed for the substrate.
- 5 Layer Distinction 1 holds operationally per A2.87 — CKS is operationally distinguishable from workflow engines.
- 6 Layer Distinction 2 holds operationally per A2.88 — CKS is operationally distinguishable from agent frameworks.
- 7 Layer Distinction 3 holds operationally per A2.89 — CKS is operationally distinguishable from control planes.

A system that fails any of (1)–(7) does not instantiate the three architectural layers at the integrated level. The individual operational tests for each layer distinction and for the composition pattern are specified in A2.87–A2.90; this test is the integrating-frame test, not a substitute for the per-specialization tests.

9. Why naming the integrating frame as standalone matters

Implementations under pressure to position CKS commercially consistently drift toward presentations that conflate CKS with one of the orchestration-layer object families. The drift is steady because the orchestration-layer objects are operationally familiar — workflow engines have decades of prior work; agent frameworks are the dominant 2024–2026 LLM-coordination pattern; control-plane primitives are well-understood through Stevens-style governance work — and CKS's distinguishing commitment operates at a different architectural layer rather than at a competing-feature level.

Implementations that drift away from the layer boundaries produce systems coherent at one layer and broken at another: commercial-positioning failures (CKS appears as a workflow enhancement or an agent-framework variant), patent-prior-art failures (the architectural commitments cannot be defensively contested when conflated with orchestration-layer prior art), composition failures (CKS-with-workflow-engine, CKS-with-agent-framework, and CKS-with-control-plane compositions become incoherent), and architectural-commitment failures (the foundational A1.01, A1.04, and A1.07 commitments are obscured when the layer distinctions are unclear).

Naming the three-architectural-layers integrating frame as a standalone architectural commitment — three layers defined in §§2–4, three layer distinctions named in §5, composition pattern named in §6, limitations in §7, operational test in §8 — gives downstream readers a precise specification of what CKS commits at and what it does not. The subsequent notes A2.87–A2.90 specialize each layer distinction and the composition pattern; together with this integrating frame, they give the full operational decomposition of A1.15 — the orchestration-layer adjacency treatment that complements A1.14's memory-location adjacency treatment to specify CKS's full architectural position in a coordination system.

Source paper

Li, W. (2026). *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems*. April 2026. ORCID: 0009-0004-8065-3235.

How to cite this note

Li, W. (2026). *The Three-Layer Architectural Model as Integrating Frame for the Layer Distinctions in the Coordination Knowledge Substrate Pattern*. May 5, 2026. ORCID: 0009-0004-8065-3235.